

電腦計時系統架構與運作原理

本文件旨在從物理層、數位邏輯層至作業系統層，系統化解析電腦時鐘之運作機制。內容涵蓋訊號震盪原理、數位電路實作細節、核心中斷處理及網路同步演算法。

1. 物理層：時脈訊號產生機制 (Physical Layer: Signal Generation)

電腦時間的根源來自於物理震盪，其核心元件為石英震盪器 (Quartz Crystal Oscillator)。

1.1 逆壓電效應與機械共振

石英晶體 (SiO_2) 具備壓電特性，其震盪機制基於以下物理定律：

- **逆壓電效應 (Inverse Piezoelectric Effect)**：對石英晶體施加電場時，晶體晶格會發生機械形變。
- **正壓電效應 (Direct Piezoelectric Effect)**：當晶體發生機械形變恢復時，表面會產生電荷 (電壓)。

運作迴路：外部電路 (Inverter Amplifier) 提供初始電壓脈衝 → 晶體形變 → 晶體回彈產生反向電壓 → 訊號回授 (Feedback) 至電路放大 → 維持持續且穩定的**機械共振 (Mechanical Resonance)**。

1.2 頻率除頻邏輯 (Frequency Divider Logic)

標準 RTC 震盪頻率定為 32.768 kHz，這是為了配合數位電路的特性。

- **T型正反器 (T Flip-Flop) 原理**：
 - **定義**：T (Toggle) Flip-Flop 是一個基本的數位記憶單元。當輸入端時脈 (Clock) 觸發邊緣 (Edge Trigger) 時，輸出狀態 (Q) 會反轉 (0變1, 1變0)。
 - **除頻效果**：因為輸出狀態每兩次 Clock 觸發才完成一個完整的 0-1-0 週期，所以 **1個 T Flip-Flop 的輸出頻率是輸入頻率的 1/2**。
- **15級串聯 (Cascaded Setup)**：
 - **級聯 (Cascaded)**：將第一級 Flip-Flop 的輸出 (Q_0) 接到第二級的時脈輸入 (CLK_1)，依此類推。
 - **計算**：串聯 15 個 T Flip-Flop，總除頻率為 $2^{15} = 32768$ 。

$$f_{out} = \frac{32768 \text{ Hz}}{2 \times 2 \times \dots (15 \text{ times})} = 1 \text{ Hz}$$

- 這產生了每秒一次的脈衝訊號，用來驅動 RTC 計數器。

2. 硬體邏輯層：RTC 與 CMOS (Hardware Logic Layer)

2.1 RTC (Real-Time Clock) 電路實作

RTC 內部並沒有 CPU 或 ALU，它是純粹的**時序邏輯電路 (Sequential Logic Circuit)**。

- **Cascaded Counters (級聯計數器)**：
 - 這是指將「秒計數器」的**溢位訊號 (Overflow/Carry)** 作為「分計數器」的**時脈輸入 (Clock Input)**。
 - 當秒針從 59 變成 00 時，產生一個 Pulse，分針計數器收到這個 Pulse 就會 +1。

- **Modulo (模數) 的電路實作：**
 - 在沒有 ALU 的情況下，Mod 運算（例如 Mod 60）是透過 **邏輯閘 (Logic Gates)** 與 **非同步重置 (Asynchronous Reset)** 實現的。
 - **實例 (Mod 10)：**使用 4 個 Flip-Flop 代表 4-bit (0000-1111)。
 1. 我們希望數到 10 (二進位 1010) 時歸零。
 2. 將代表 Bit 3 (Q_3) 和 Bit 1 (Q_1) 的輸出線連接到一個 **NAND Gate**。
 3. 當數值變成 1010 的瞬間， $Q_3 = 1, Q_1 = 1$ ，NAND Gate 輸出低電位訊號。
 4. 此訊號連接到所有 Flip-Flop 的 **CLR (Clear/Reset)** 腳位。
 5. 電路瞬間被強制重置為 0000。
 - 這就是硬體層次的 Mod 運算，完全不需要軟體介入。

2.2 I/O 存取機制 (I/O Port Access)

CPU 如何讀取這些計數器的值？

- **x86 I/O 架構：**RTC 透過 **I/O Port 0x70 & 0x71** 進行溝通。
- **Index/Data 模式 (Indirect Addressing)：**
 - 為了節省 I/O Address Space，RTC 只佔用兩個 Port。
 - **OUT 0x70, <Index>**：這是「地址暫存器」。CPU 告訴 RTC：「我要存取內部第幾號暫存器（例如 0x00 是秒，0x02 是分）」。這就像打電話撥分機號碼。
 - **IN 0x71**：這是「數據暫存器」。RTC 內部電路根據剛才收到的 Index，將對應暫存器的資料放上 Data Bus，CPU 再讀取進來。

2.3 CMOS RAM 與 BIOS 的區別

- **ROM vs RAM：**
 - **BIOS Code (Firmware)：**存放在 **Flash ROM** (SPI Flash) 中。這是唯讀（或需特定程序寫入）的程式碼，包含 POST 程序、硬體初始化代碼。
 - **BIOS Settings (Configuration)：**存放在 **CMOS RAM** 中。這是使用者的偏好設定（如開機順序、系統時間、超頻參數）。
- **為什麼分開？**
 - ROM 無法隨意更改，且寫入速度慢。
 - RAM 可以快速讀寫，方便使用者修改設定。但 RAM 斷電會掉資料，所以才需要電池。
- **Bootstrapping 關聯：**CPU 通電 → 跳轉 Reset Vector (Flash ROM) → 執行 POST → **讀取 CMOS RAM (得知硬體設定與時間)** → 根據設定去載入 MBR/EFI Loader。

2.4 CMOS 低功耗物理原理

為何強調 CMOS (Complementary MOS)？這解釋了電池壽命。

- **CMOS 結構：**由 **PMOS (上拉網路, Pull-up)** 和 **NMOS (下拉網路, Pull-down)** 互補組成。
- **靜態狀態 (Static State)：**
 - 意指電路處於「保持 0」或「保持 1」，且**時脈沒有在跳動的狀態**。

- 若輸出為 High (1)：PMOS 導通，NMOS 截止 (Cut-off)。電流路徑從 Vdd 到 Output，但無法流向 Ground。
- 若輸出為 Low (0)：NMOS 導通，PMOS 截止。電流路徑從 Output 到 Ground，但 Vdd 端被切斷。
- **理論電流為 0：**
 - 根據歐姆定律，電路中必須形成迴路 (Vdd to Gnd) 才有電流。
 - 在 CMOS 的靜態邏輯中，**永遠有一個電晶體是關閉的**，切斷了 Vdd 到 Gnd 的直通路徑。
 - 因此 $I_{static} \approx 0$ 。實際上只有奈安培 (nA) 等級的漏電流 (Leakage)。
 - **對比：**早期的 TTL (BJT) 或 NMOS 邏輯，即使在維持狀態，電阻性負載也會持續消耗電流。

3. 系統架構層：時脈生成與分配 (System Architecture: Generation & Distribution)

RTC 僅提供低頻 (32.768 kHz) 的人類時間，但 CPU 需要 GHz 等級的高頻訊號。此層級負責「倍頻」與「全域分配」。

3.1 鎖相迴路 (PLL, Phase-Locked Loop) 深度解析

PLL 不僅是電路，它是經典控制理論 (Classic Control Theory) 的完美體現。我們在此深入拉普拉斯域 (s-domain) 的物理意義。

3.1.1 什麼是相位 (Phase) 與 弧度 (Radian)？

在討論 PLL 之前，必須嚴格定義「我們到底在鎖定什麼」。

- **弧度 (Rad)：**這是一種角度的度量單位，定義為「圓弧長度等於半徑時的張角」。 $360^\circ = 2\pi \text{ rad}$ 。使用弧度是因為在微積分中， $d(\sin x)/dx = \cos x$ 只有在 x 為弧度時才成立，這簡化了所有頻率分析。
- **相位 (θ)：**波形在週期運動中的「當前位置」。
 - 想像一個跑道（時鐘週期），**頻率 (ω)** 是車速（每秒跑幾圈，rad/s）。
 - **相位 (θ)** 是車子現在在跑道的哪個位置（跑了幾圈又幾度，rad）。
- **物理關係：**頻率是相位的變化率。

$$\omega(t) = \frac{d\theta(t)}{dt} \iff \theta(t) = \int \omega(t) dt$$

- **關鍵意義：**這意味著**相位是頻率的積分**。這一點是 PLL 能夠實現「零頻率誤差」的數學根源。

3.1.2 為什麼要有 PD -> Filter -> VCO 這三個步驟？

我們不能直接給 VCO 一個 Offset，原因如下：

1. **VCO 不完美：**VCO 的特性曲線 $f_{out} = f_0 + K_v V_{ctrl}$ 會隨溫度、電壓漂移。你給固定的 Offset，輸出頻率就會隨環境亂飄。
2. **閉環控制 (Closed Loop)：**我們需要一個機制去「看」輸出對不對。
 - **Phase Detector (PD)：**測量誤差。比較參考訊號與 VCO 輸出的相位差。
 - **Loop Filter (LF)：**決策與平均。濾除高頻雜訊，計算出需要的修正力道。

- **VCO：執行修正**。調整頻率以消除誤差。

3.1.3 s-domain (拉普拉斯域) 的意義與轉換

我們使用拉普拉斯轉換 (L) 將微分方程式轉換為代數方程式。

- **轉換意義：**
 - **時間域 (t)**：處理 d/dt (微分) 和 $\int dt$ (積分)，計算複雜。
 - **頻率域 (s)**： s 是一個複數變數 ($s = \sigma + j\omega$)。在 s-domain 中，微分變成乘以 s ，積分變成除以 s 。
 - **實際意義**：這讓我們可以直接用「乘法」來設計系統的穩定性（看極點 Pole 的位置）和頻率響應，而不用解微分方程。

3.1.4 各組件轉移函數與 K_p 計算

我們在 s-domain 建立模型：

1. **相位檢測器 (PD) 與 K_p ：**
 - **功能**：將相位差 θ_e (rad) 轉成電壓 V_{pd} (Volt)。
 - **K_p (Gain)**：這是 PD 特性曲線的斜率，單位是 V/rad 。
 - **電路實作與計算：**
 - 以 **XOR Gate** 為例：當兩訊號相位差從 0 變到 π (180度) 時，輸出佔空比從 0% 變到 100%，平均電壓從 0 變到 V_{DD} 。
 - 因此： $K_p = \frac{\Delta V}{\Delta \theta} = \frac{V_{DD}-0}{\pi-0} = \frac{V_{DD}}{\pi}$ (Volts/rad)。
 - **轉移函數**： $V_{pd}(s) = K_p \cdot (\theta_{ref}(s) - \theta_{fb}(s))$ 。
2. **壓控振盪器 (VCO) 與 $1/s$ (積分器的深刻物理意義)：**
 - **數學事實**：VCO 輸入是電壓(頻率)，輸出是相位。因為 $\theta = \int \omega dt$ ，在 s-domain 對應 $1/s$ 。

$$\frac{\Theta_{out}(s)}{V_{ctrl}(s)} = \frac{K_v}{s}$$

- **為什麼「穩態誤差為零」？(Why Zero Steady-State Error?)** 這裡需要引入控制理論中的三個關鍵變數定義與 **終值定理 (Final Value Theorem)**。
- **A. 開迴路轉移函數 ($G(s)$, Open-Loop Transfer Function)**：這是訊號在「前向路徑」上經過的所有組件的增益總和，沒有回授。在此 PLL 模型中，路徑是 PD $\rightarrow$ Filter $\rightarrow$ VCO。

$$G(s) = K_p \cdot H_f(s) \cdot \frac{K_v}{s}$$

注意：這裡分母有一個 s (來自 VCO)，這代表 $G(s)$ 擁有一個原點極點，系統類型為 **Type 1**。

- **B. 誤差函數 ($E(s)$, Error Function)**：這是我們最關心的「差距」，即參考輸入 $R(s)$ 與回授訊號的差值。根據負回授公式： $E(s) = \frac{1}{1+G(s)} \cdot R(s)$ 。其中 $R(s)$ 是輸入訊號（參考相位）。
- **C. 穩態誤差 (e_{ss} , Steady-State Error)**：這是當時間趨近於無窮大 ($t \rightarrow \infty$) 時，誤差 $e(t)$ 最終殘留的值。如果 $e_{ss} = 0$ ，代表系統完美追蹤了目標。

- **數學證明 (Proof)：** 假設輸入是一個**階梯訊號 (Step Input)**，即相位突然跳變，或者我們只想維持固定相位差。在 s-domain 中，階梯輸入 $R(s) = \frac{1}{s}$ 。利用 **終值定理**：
 $\lim_{t \rightarrow \infty} e(t) = \lim_{s \rightarrow 0} s \cdot E(s)$ 。將 $R(s)$ 和 $G(s)$ 代入：

$$e_{ss} = \lim_{s \rightarrow 0} s \cdot \frac{1}{1 + G(s)} \cdot \frac{1}{s} = \lim_{s \rightarrow 0} \frac{1}{1 + G(s)}$$

觀察 $G(s)$ 的表示式： $G(s) = \frac{K_p H_f(s) K_v}{s}$ 。當 $s \rightarrow 0$ 時，因為分母趨近於 0，所以 $G(s)$ 的極限值趨近於無窮大：

$$\lim_{s \rightarrow 0} G(s) = \infty$$

代回穩態誤差公式：

$$e_{ss} = \frac{1}{1 + \infty} = 0$$

物理結論： 只要迴路中存在一個完美的積分器 ($1/s$)，在直流（穩態）情況下，開迴路增益會趨近無窮大。這意味著哪怕只有極微小的誤差，也會被系統放大，迫使輸出進行修正，直到誤差完全歸零。這就是為什麼 PLL 鎖定後，頻率完全相等，沒有任何穩態誤差。

3.1.5 RC 濾波器與積分作用 (Loop Filter)

這是將相位誤差轉化為穩定控制訊號的關鍵。

- **RC 濾波器：** 由電阻 (R) 和電容 (C) 組成的電路。
 - **物理電路：** 輸入訊號先經過電阻 R，然後並聯一個電容 C 接地。
 - **轉移函數：** $H_f(s) = \frac{1}{1 + sRC}$ （無源低通）。
- **積分作用 (Integration Effect)：**
 - **物理意義：** 電容具有「儲存電荷」的能力 ($Q = C \times V$)。電流流入電容時，電壓不會瞬間跳變，而是隨著電荷累積而線性或指數上升。公式： $V(t) = \frac{1}{C} \int i(t) dt$ 。
 - **為什麼需要它？**
 - **記憶效應：** 電容「記住了」過去所有的誤差累積。只要相位誤差還存在（PD 輸出電壓），電容就會繼續充電， V_{ctrl} 就會持續改變，逼迫 VCO 去消除那個誤差。
 - **濾波：** PD 的輸出通常是含有高頻成分的脈衝波（PWM），RC 電路能濾掉這些「毛刺」，只留下平均的直流電壓來平穩控制 VCO。
- **跟蹤頻率與加速度：** 如果採用**主動式濾波器 (Active Filter)**，轉移函數包含 $\frac{1}{sRC}$ （純積分器）。
 - **Type 2 System：** 系統中有了兩個積分器（VCO 一個 $1/s$ ，濾波器一個 $1/s$ ）。
 - **物理結果：** 這保證了對於**斜坡輸入 (Ramp Input)**，也就是「頻率有差異」的情況，系統能達到**零穩態誤差**。這就是 PLL 不只能鎖相位，還能完美鎖定頻率的數學保證。

3.1.6 系統動力學：阻尼與臨界阻尼 (Damping)

PLL 是一個二階動態系統，其行為可以用彈簧-阻尼系統來類比。

- **阻尼 (Damping Ratio, ζ)：** 描述系統受到擾動後，能量耗散並回歸穩定的能力。
 - **欠阻尼 (Under-damped, $\zeta < 1$)：**

- **現象：**系統反應很快，但會發生**震盪 (Ringing)**。頻率在鎖定前會上下衝過頭 (Overshoot) 好幾次。
- **缺點：**時鐘不穩定，容易導致時序錯誤。
- **過阻尼 (Over-damped, $\zeta > 1$)：**
 - **現象：**系統像在泥沼中移動，反應非常**遲鈍 (Sluggish)**。
 - **缺點：**鎖定時間 (Lock Time) 太長，開機或喚醒變慢。
- **臨界阻尼 (Critical Damping, $\zeta = 1$ 或工程上常取 0.707)：**
 - **定義：**這是數學上的「甜蜜點」。系統以**最快的速度**回到目標值，且剛好不發生震盪。
 - **為什麼是 0.707 ($\frac{1}{\sqrt{2}}$)？**
 - 在頻率響應上， $\zeta = 0.707$ 能提供最平坦的通帶響應 (Butterworth Filter 特性)，沒有頻率凸起 (Peaking)，同時保證了良好的瞬態響應速度。這對於 CPU 時鐘產生器來說是最佳設計參數。

3.1.7 閉環公式總結

結合上述所有組件，完整的閉環轉移函數為：

$$H(s) = \frac{\theta_{out}(s)}{\theta_{ref}(s)} = \frac{K_p H_f(s) \frac{K_v}{s}}{1 + K_p H_f(s) \frac{K_v}{s}}$$

這個公式是控制工程師設計 PLL 頻寬、鎖定時間與穩定度的核心依據。

3.2 時鐘樹分配 (Clock Distribution Tree)

時鐘訊號生成後，必須像血管一樣輸送到晶片上數十億個電晶體。

- **拓撲結構：**為了確保所有邏輯單元在「同一時間」收到時鐘訊號，避免**時鐘偏斜 (Clock Skew)**，採用樹狀結構：
 - **H-Tree：**遞歸的 H 型結構，保證從根節點到每個葉節點的路徑長度相等。
 - **全域與區域緩衝 (Global/Local Buffers)：**多級緩衝器 (GCLK -> First Stage -> Second Stage) 推動大負載。
- **圖論優化 (Steiner Tree Problem)：**時鐘樹佈線本質上是求解 **最小 Rectilinear Steiner 樹 (RSMT)** 問題。
 - **目標：**連接所有終端 T ，引入中間節點 (Steiner Points)，使總線長與 Skew 最小化。
 - **代價函數：** $Cost = \sum WireLength + \alpha \sum BufferArea + \beta \cdot Skew$ 。
 - **約束：**現代高頻 CPU 要求 $Skew < 50 \text{ ps}$ 。

3.3 邏輯電平標準 (Logic Levels: TTL vs CMOS)

時鐘與數據訊號的高低電位判斷，取決於製造工藝的閾值。

- **TTL (Transistor-Transistor Logic)：**使用 BJT。 $V_{CC} = 5V$ 。
 - 邏輯 0： $0 \sim 0.8V$ 。
 - 邏輯 1： $2.0 \sim 5V$ 。
 - **雜訊邊際 (Noise Margin)：**低，功耗較高。

- **CMOS (Complementary MOS)**：使用 MOSFET。
 - **閾值電壓 (V_{th})**：約為 $V_{DD}/2$ （例如 3.3V CMOS 閾值約 1.65V）。
 - **優勢**：高雜訊邊際，靜態功耗極低，適合現代 VLSI 設計。

4. 數據傳輸與匯流排層 (Data Transmission & Bus Layer)

CPU 透過匯流排存取記憶體或 I/O，其運作嚴格依賴時鐘同步。

4.1 同步設計 (Synchronous Design)

所有數據傳輸都以全域時鐘 (Global Clock) 的邊緣為基準。

- **時序約束**：

$$t_{prop} + t_{logic} + t_{setup} \leq T_{clock}$$

數據必須在下一個時鐘邊緣到來前的 **建立時間 (Setup Time, t_{setup})** 內穩定下來，否則會產生亞穩態 (Metastability)。

4.2 位址線與數據線 (Address & Data Bus)

為了節省引腳，許多架構（如 8085/8086）採用 **多工 (Multiplexing)** 設計，即位址線與數據線共用物理引腳 (ADO-ADn)。

- **ALE (Address Latch Enable)** 機制：這是一個關鍵的時序控制訊號，用於區分匯流排上跑的是「位址」還是「數據」。
 1. **T1 週期 (ALE = High)**：
 - CPU 在 AD 線上輸出 **位址**。
 - 外部鎖存器 (Latch, 如 74LS373) 在 ALE 下降緣瞬間鎖存位址。
 2. **T2 週期 (ALE = Low)**：
 - AD 線切換功能，用於傳輸 **數據**。
 - 因為位址已被鎖存，記憶體仍知道要存取哪裡，CPU 可安全地進行讀寫。

5. 核心層：作業系統與電源管理 (Kernel Layer: OS & Power)

作業系統不僅是被動計時，更主動參與時脈控制以管理功耗。

5.1 核心時間結構

- `struct timespec`：

```
struct timespec {
    time_t  tv_sec;    // Seconds (Unix Timestamp)
    long    tv_nsec;   // Nanoseconds
};
```

OS 在開機時讀取 RTC 初始化此結構，隨後利用 TSC (Time Stamp Counter) 或 APIC Timer 的中斷 (IRQ) 累加 `jiffies` 來維護牆上時間 (Wall Time)。

5.2 DVFS (Dynamic Voltage and Frequency Scaling)

這是一個結合 OS 排程器與底層 PLL 的閉環控制系統。

- **物理原理：**CMOS 動態功耗 $P \propto C \cdot V^2 \cdot f$ 。
 - 降低頻率 f 只能線性降低功耗。
 - 降低電壓 V 可以平方級降低功耗。
- **運作流程：**
 1. **監控：**OS 偵測到 CPU 負載降低 (Idle)。
 2. **決策：**Governor (如 Linux 的 `cpufreq`) 決定降頻。
 3. **執行：**
 - 寫入暫存器調整 PLL 的分頻比 $N \rightarrow f_{new} = f_{ref} \times N'$ 。
 - 同時通知電壓調節模組 (VRM) 降低供電電壓。
 4. **結果：**在不影響效能的前提下大幅省電。

6. 網路層：同步控制理論 (Network Layer: Synchronization)

為了解決石英震盪器的物理溫漂，OS 實作了軟體鎖相迴路。

6.1 NTP 與軟體 PLL

- **雙向延遲算法：**透過 UDP Port 123 計算網路往返時間 δ 與時間偏差 θ 。
- **頻率補償 (Frequency Compensation)：**使用 `adjtimex` 系統呼叫微調系統滴答 (Tick) 的長度 (Slew)。這本質上是在軟體層面實作一個 PLL，調整軟體時鐘的「頻率」來鎖定外部原子鐘的相位。

6.2 PTP (IEEE 1588)

對於高頻交易 (HFT) 等需要奈秒級精度的場景，採用 PTP。

- **硬體打戳 (Hardware Timestamping)：**網卡 (NIC) PHY 層直接記錄封包進出時間，繞過 OS 中斷延遲與 Context Switch 影響，實現 μs 級同步。

7. 總結：全系統邏輯鏈

1. **物理層：**壓電效應產生 32.768kHz 與 14.318MHz 基準。
2. **硬體層：**RTC 電路除頻計數，電池維持；CMOS 邏輯確保低靜態功耗。
3. **架構層：**PLL 利用負回授控制將基準訊號倍頻至 GHz；Steiner Tree 演算法將時鐘無偏斜地分配至全晶片。
4. **匯流排層：**ALE 訊號控制位址/數據多工，確保同步傳輸的建立與保持時間。
5. **核心層：**OS 維護軟體時間，並透過 DVFS 動態調整 PLL 與電壓以平衡效能與功耗。

• 硬體層：晶振提供基準頻率，PLL 鎖相倍頻至 GHz，VRM 調節電壓。